

Section 14.1: Classic Interview Problems

Part 1 of 8 in the System Design Interview Series

Executive Overview

These foundational problems appear in nearly every system design interview. Mastery requires understanding both simple and advanced solutions, plus knowledge of the trade-offs that separate junior from senior-level responses. For each system, you must be ready to justify every choice — a strong candidate explains *why*, not just *what*.

1. Design a URL Shortener (TinyURL / Bitly)

Requirements Analysis

Dimension	Requirement
Functional	Shorten long URLs; redirect short → long
Scale	100M links, 1B clicks/day
Non-functional	< 50ms redirect latency, high availability, click analytics

Read/Write ratio: ~10:1 (reads dominate — prioritize read path)

High-Level Architecture

```
Client → Load Balancer → API Servers → Cache (Redis) → MySQL
                                   ↓
                                   (Cache miss only)
```

- **API Servers** handle creation and redirect logic
- **Cache** holds hot short → long URL mappings (LRU eviction)
- **MySQL** is the source of truth; sharded by `short_hash`

Key Design Decision: Short URL Generation

Four approaches, each with distinct trade-offs:

Approach	How It Works	Generation Speed	Collision Risk	Predictability
Auto-increment	<code>MAX(id)+1</code> encoded in Base62	Fast	None	High (guessable — security risk)
Hash	<code>MD5(long_url)</code> → take first 8 chars	Fast	Medium (birthday paradox)	Low
Random	8-char random Base62 string + collision check	Medium	Low	Low
Snowflake-style	Timestamp + node ID + sequence	Fast	None (with worker IDs)	Leaks timestamp

Recommended: Random with collision check for most use cases. Hash is fine but requires a retry loop. Auto-increment leaks business metrics.

When to use Hash: When the same long URL must always produce the same short URL (idempotent creation). Requires explicit collision handling.

When to use Snowflake-style: When you need audit/tracing (you can extract creation time from the ID). Not appropriate when users shouldn't infer creation order.

Storage Schema

```
CREATE TABLE urls (  
  short_hash VARCHAR(16) PRIMARY KEY,  
  long_url   TEXT      NOT NULL,  
  created_at TIMESTAMP,  
  expires_at TIMESTAMP,  
  user_id    BIGINT,  
  click_count INT      DEFAULT 0  
);  
  
CREATE INDEX idx_user_urls ON urls(user_id);
```

Sharding strategy: Shard by `short_hash` (range or consistent hash). User-based queries (`idx_user_urls`) will require scatter-gather across shards — acceptable since they're lower frequency.

Failure Modes & Mitigations

Failure	Root Cause	Mitigation
Hot keys	Popular short URLs overwhelm single cache node	CDN-level redirect cache; request coalescing
Hash collisions	Two different long URLs → same short hash	Check before insert; retry with new random hash
Cache stampede	Expiry triggers simultaneous DB reads	Staggered TTL; probabilistic early expiry
Single point of failure	One DB node	Primary-replica with automatic failover

Interview Problem 1 — URL Shortener Variant

“Design a URL shortener that supports per-user custom aliases (e.g., `/my-brand`) alongside auto-generated hashes. How do you prevent alias collisions and abuse?”

Solution framework:

1. **Namespace isolation:** Store custom aliases in the same table but with a `type` column (`CUSTOM` vs `AUTO`). The `short_hash` column is the lookup key regardless.
2. **Conflict detection:** On alias creation, `INSERT INTO urls ... ON CONFLICT DO NOTHING` . If 0 rows inserted, alias is taken — return 409 Conflict.
3. **Abuse prevention:** Custom aliases require auth. Rate-limit alias creation per user (e.g., 100/day). Block reserved words (`api` , `admin` , `health`).
4. **Length constraints:** Enforce 3–50 chars, alphanumeric + hyphens only, validated at API layer before hitting DB.

2. Design a Social Media News Feed (Facebook / Twitter)

Requirements Analysis

Dimension	Requirement
Functional	Post content; read personalized home timeline
Scale	100M users, 10M DAU, 1B posts/day
Fan-out challenge	Average user follows 500; celebrities follow millions

Key insight: The hard problem is not storage — it's the fan-out asymmetry between normal users and celebrities.

Fan-out Approaches Compared

Approach	Write Cost	Read Cost	Consistency	Celebrity Support
Fan-out-on-write	$O(\text{followers})$ — very high for celebrities	$O(1)$ — pre-built cache	Strong (near real-time)	Poor — 10M writes per tweet
Fan-out-on-read	$O(1)$	$O(\text{following count})$ — slow at scale	Eventual	Excellent
Hybrid	Low for celebrities; high for normal users	Low for both	Mixed	Good

Recommended production approach: Hybrid

Normal user posts ($\leq 10K$ followers):

→ Fan-out-on-write: push to each follower's timeline cache (Redis sorted set)

Celebrity posts ($> 10K$ followers):

→ Fan-out-on-read: store centrally; merge into timeline at read time

Timeline read:

→ Fetch pre-built timeline from Redis

→ Merge with celebrity posts from central store

→ Rank by recency or engagement score

Failure Modes & Mitigations

Failure	Mitigation
Write amplification (celebrity tweets)	Queue-based fan-out; rate-limit fan-out workers
Cache stampede (timeline TTL expires)	Staggered TTL per user; background refresh
Follower list too large to fetch	Paginated fan-out in batches of 1,000; async queue

Interview Problem 2 — News Feed Ranking

“Your product team wants to replace chronological feed ordering with a ranked feed (like Instagram’s algorithm). How do you implement this without increasing latency?”

Solution framework:

1. **Offline scoring:** Run a batch ML job hourly that produces per-user affinity scores for each creator they follow. Store in Redis: `affinity:{user_id}:{creator_id} → score`.
2. **At read time:** Fetch candidate posts from timeline cache → retrieve affinity scores → compute `final_score = recency_decay * affinity_score * engagement_weight` → sort and return top K.
3. **Latency budget:** Affinity lookup is a Redis MGET (single round-trip). Sorting K=200 candidates in memory is microseconds. Total overhead ≤ 5ms.
4. **Cold start:** New users have no affinity scores — fall back to chronological or popularity-based ranking until enough signal accumulates.

3. Design a Typeahead / Autocomplete Service

Requirements Analysis

Dimension	Requirement
Functional	Real-time suggestions as user types
Scale	10M queries/minute at peak
Latency	< 50ms for top-10 suggestions

Data Structure Comparison

Structure	Lookup Speed	Update Speed	Memory	Best For
Trie (Prefix Tree)	$O(\text{prefix_len} + K)$	$O(\text{word_len})$	High	Exact prefix match, custom completions
Redis Sorted Set	$O(\log N + K)$	$O(\log N)$	Medium	Distributed, simple deployment
Search Engine (ES)	$O(\text{query})$	Batch/near-real-time	High	Multi-field, fuzzy matching, boosting

Production architecture:

- **Hot prefixes (top 10K)**: Pre-warmed in Redis sorted sets. `ZRANGEBYLEX "amaz" "[amaz" "[amazx" LIMIT 0 10`
- **Tail prefixes**: Fall through to Trie service or Elasticsearch
- **Update pipeline**: Aggregated query logs → Hadoop batch job → update suggestion weights → push to Redis/Trie nightly

Trie Example

Prefix tree for "amazon", "amazing", "amaze":

```
a → m → a → z → o → n (weight: 50M searches)
      ↘ e (weight: 5M searches)
        ↘ i → n → g (weight: 20M searches)
```

Each node stores `(char, children[], frequency)`. Top-K at a prefix node is maintained as a small heap updated during bulk reweighting.

Interview Problem 3 — Personalised Autocomplete

“Google Search shows different suggestions to different users based on their history. How do you design personalised autocomplete at scale?”

Solution framework:

1. **Two-layer suggestion**: Global suggestions (same for all) + personal suggestions (user-specific). Merge and deduplicate at serve time.
2. **Personal layer storage**: `personal_suggestions:{user_id}` → Redis sorted set of (query, recency-weighted score). Max 500 entries per user (LRU eviction).
3. **Merge strategy**: Interleave global top-5 and personal top-5, deduplicating and boosting personal matches that appear in both lists.
4. **Privacy**: Personal suggestions stored only server-side with encryption at rest. Users can clear via settings (DELETE all keys for user_id).
5. **Latency**: Two parallel Redis lookups → merge in application layer → total < 10ms.

4. Design a Distributed Key-Value Store (Dynamo / Cassandra Style)

Requirements Analysis

Dimension	Requirement
Scale	Planet-scale, 100+ nodes
Availability	AP system — survive partitions
Consistency	Eventual; tunable via quorum ($R + W > N$)

Core Mechanics

Consistent Hashing Ring

Each key is hashed (MD5 or SHA-1) to a position on a virtual ring $[0, 2^{32})$. Nodes occupy positions on the ring. A key's primary node is the first node clockwise from the key's position. Virtual nodes (vnodes) smooth load distribution:

Ring: Node A (0°) ... Node B (120°) ... Node C (240°) ...

Key hash = 90° → assigned to Node B (first clockwise)

Replication: also write to Node C, Node A (next N-1 nodes clockwise)

Quorum Math — Give extra weight to this:

With $N=3$ replicas: - $W=2$ (write quorum), $R=2$ (read quorum): $W+R=4 > N=3 \rightarrow$ Strong consistency guaranteed - $W=1, R=1$: $W+R=2 \leq N \rightarrow$ Eventual consistency, maximum availability - $W=3, R=1$: All replicas must ack write — maximum durability, lower write availability

Proof of correctness: If $W+R > N$, at least one node in every read quorum overlaps with every write quorum. That node holds the most recent write, so stale reads are impossible. When $W+R \leq N$, the sets can be disjoint — a read can miss the latest write.

Vector Clocks

Each value is versioned with a vector clock: `[NodeA: 2, NodeC: 1]` means NodeA incremented its counter twice, NodeC once.

- **Merge rule:** Take element-wise max across all vector clocks when reconciling
- **Conflict detection:** If neither VC dominates the other (incomparable), a conflict exists — expose both versions to the application for semantic resolution
- **Cleanup:** Delete old VC entries when all nodes have seen a later version

Failure Modes & Mitigations

Failure	Mechanism	Mitigation
Node goes down during write	Write partially replicated	Hinted handoff: another node accepts write with hint to forward when target recovers
Stale replica	Read returns old data	Read repair: coordinator detects divergence during quorum read, asynchronously repairs stale replicas
Replica falls far behind	Long outage	Anti-entropy via Merkle tree: compare subtrees to find divergent key ranges efficiently ($O(\log N)$ comparison instead of $O(N)$)

5. Design a Distributed Cache (Redis / Memcached)

Caching Strategy Comparison

Strategy	How It Works	Consistency	Write Latency	Risk	Best For
Cache-aside (lazy loading)	App reads cache; on miss, reads DB and populates cache	Weak (stale until TTL)	Variable (miss penalty)	Low	Read-heavy, simple scenarios
Read-through	Cache fetches from DB on miss transparently	Good	Consistent	Low	Abstracted cache clients
Write-through	App writes to cache AND DB synchronously	Strong	Higher	Low	Strong consistency needs
Write-behind (write-back)	App writes to cache; async flush to DB	Weak	Lowest	High (data loss on crash)	Throughput-critical, tolerable data loss

When to use write-behind: Session stores, counters, ephemeral analytics. Never for financial data or inventory.

When to use write-through: User profile updates, configuration data — where reads must reflect latest write immediately.

Cache Cluster Architecture

Clients → Consistent Hash (Ketama) → [Redis Node 1] [Node 2] ... [Node N]

- **Consistent hashing** minimises cache invalidation when nodes are added/removed (only K/N keys remapped vs all keys)
- **Replication:** Redis Cluster uses primary + replica per shard; automatic failover via Sentinel

6. Design a Distributed Rate Limiter

Algorithm Comparison

Algorithm	Accuracy	Memory Cost	Scalability	Best For
Token Bucket	Approximate (burst allowed)	$O(1)$ per user	Excellent	API gateways, burst-tolerant limits
Fixed Window Counter	Poor (boundary burst problem)	$O(1)$ per user	Excellent	Coarse-grained limits
Sliding Window Log	Perfect	$O(\text{requests})$ per user	Poor	Low-volume, strict enforcement
Sliding Window Counter	Good (~1% error)	$O(1)$ per user	Excellent	Production default

Boundary burst problem with Fixed Window: A user can send $2 \times \text{limit}$ requests by hitting the end of window N and start of window N+1 in quick succession. Sliding window eliminates this.

Production Implementation (Sliding Window Counter)

```
def rate_limit(user_id, limit=100, window=60):
    now = int(time.time())
    current_window = now // window
    key_cur = f"rate:{user_id}:{current_window}"
    key_prev = f"rate:{user_id}:{current_window - 1}"

    # Atomic Lua script
    result = redis.eval("""
        local cur = tonumber(redis.call('GET', KEYS[1]) or 0)
        local prev = tonumber(redis.call('GET', KEYS[2]) or 0)
        local elapsed_fraction = tonumber(ARGV[2])
        local weighted = prev * (1 - elapsed_fraction) + cur
        if weighted < tonumber(ARGV[1]) then
            redis.call('INCR', KEYS[1])
            redis.call('EXPIRE', KEYS[1], tonumber(ARGV[3]))
            return 1
        end
        return 0
    """, 2, key_cur, key_prev, limit, (now % window) / window, window * 2)
    return bool(result)
```

Interview Problem: Global Gaming Rate Limiter

“Design a rate limiter for a global gaming platform generating 10M RPS. How do you achieve near-global consistency without prohibitive latency?”

Solution framework:

1. **Hierarchical limits:** Per-user hard limit enforced locally (Redis in same AZ, < 1ms). Per-region soft limit. Global probabilistic limit via Count-Min Sketch.
2. **Probabilistic global counting:** Count-Min Sketch provides approximate global count with configurable error rate (< 2%). Accept 2–5% over-allocation rather than requiring cross-region consensus.
3. **Async sync:** Each region asynchronously reports its counters to a global aggregator every 100ms. Aggregator updates regional budgets.
4. **Emergency override:** If global count exceeds 110% of limit (detected by aggregator), broadcast hard-stop signal to all regions.

Key insight: CAP theorem applies to rate limiting. You cannot have perfect consistency, perfect availability, and zero latency simultaneously at 10M RPS. Probabilistic enforcement + hard per-region limits gives practical correctness with acceptable over-allocation.

7. Design a Distributed Unique ID Generator (Snowflake)

Requirements Analysis

Dimension	Requirement
Uniqueness	No collisions across all services globally
Sortability	IDs roughly time-ordered (for DB index efficiency)
Scale	100K+ IDs/second globally

Snowflake 64-bit Layout

```
Bit 63 (sign): always 0
Bits 22-62 (41 bits): milliseconds since custom epoch → supports ~69 years
Bits 12-21 (10 bits): worker ID (datacenter 5 bits + machine 5 bits) → 1024 workers
Bits 0-11 (12 bits): sequence number → 4096 IDs per ms per worker
```

Throughput math: 4096 IDs/ms × 1024 workers = ~4.2B IDs/ms globally.

ID Format Comparison

Format	Sortable	Collision Risk	Info Leakage	Size	Best For
UUID v4	No	None	Minimal	128 bits	Simple uniqueness, no ordering needed
Snowflake	Yes (ms precision)	None (with worker IDs)	Timestamp + node ID	64 bits	High-throughput ordered IDs
ULID	Yes (ms precision)	None	Timestamp only	128 bits	URL-safe Snowflake alternative

When timestamps in IDs are a problem: In competitive gaming or auctions, exposing creation time allows inference of activity patterns. Use ULID (random suffix) or obfuscate with XOR.

Failure Modes

Failure	Cause	Mitigation
Clock drift backward	NTP correction or hardware issue	Reject IDs with <code>timestamp < last_timestamp</code> ; wait until clock catches up
Sequence overflow	> 4096 IDs in same ms	Busy-wait until next millisecond; monitor and alert if this is frequent
Worker ID collision	Two machines assigned same ID	Use ZooKeeper or etcd to coordinate worker ID assignment at startup

Section Summary: Choosing the Right Classic Pattern

Problem Type	Key Concern	Go-To Pattern
URL Shortener	Read latency, hot keys	Cache-aside + CDN
News Feed	Fan-out at scale	Hybrid fan-out (write for normal, read for celebrities)
Autocomplete	Prefix query speed	Trie for exact match; ES for fuzzy
KV Store	CAP trade-off	Tunable quorum (Dynamo-style)
Rate Limiter	Accuracy vs latency	Sliding window counter + Redis Lua
Unique IDs	Ordering + throughput	Snowflake (64-bit)